

Mathematical logic

(3) Implementation design of SAT solver and
abstract DPLL

Normal form

- **Def. Conjunctive normal form, CNF**

- ✓ Clauses (= sequences of literals connected by $\vee$) are connected by $\wedge$.

- ✓ $(A_1 \vee \neg A_2 \vee A_3) \wedge (B_1 \vee B_2 \vee B_3) \wedge (\neg C_1 \vee C_2)$

- **Def. Disjunctive normal form, DNF**

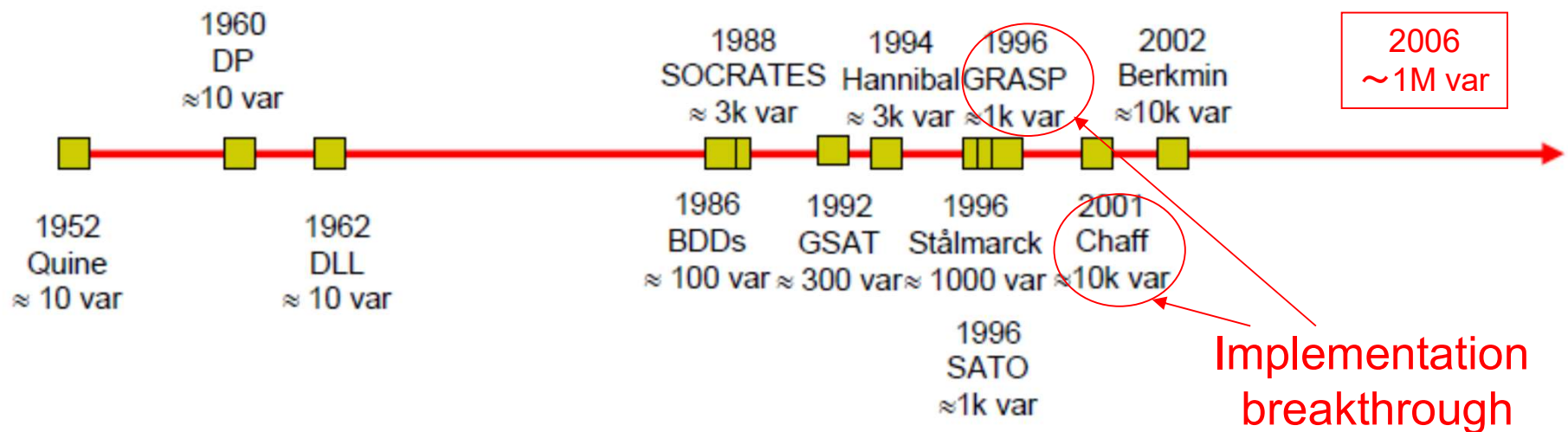
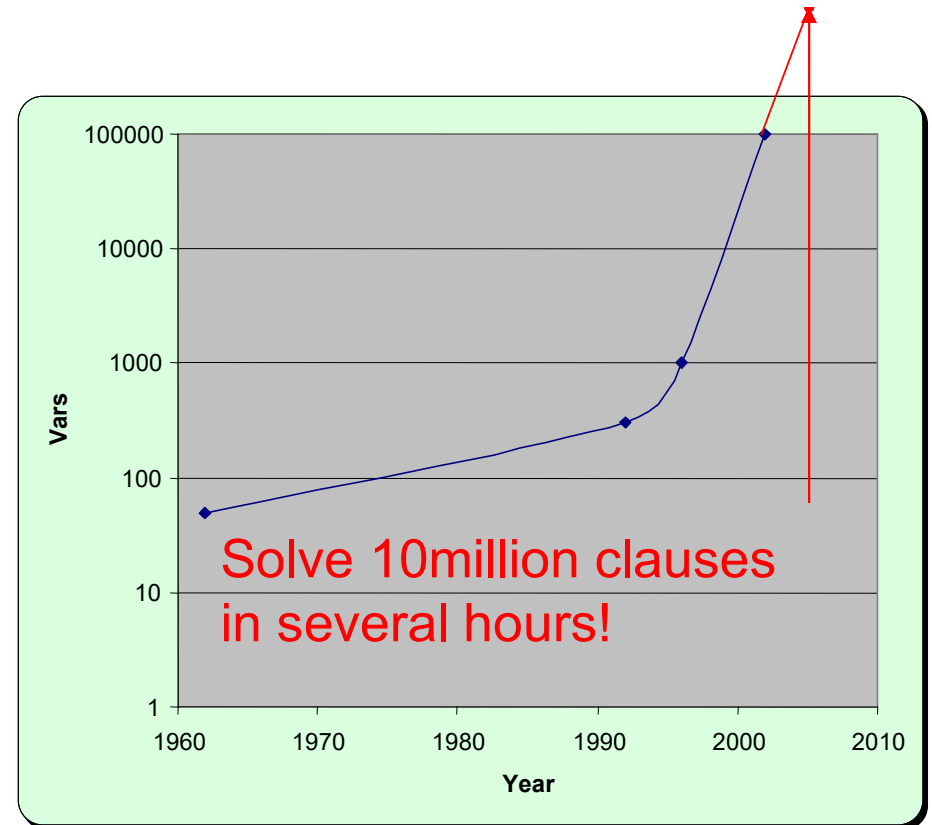
- ✓ Sequences of literals connected by $\wedge$ are connected by $\vee$.

- ✓ $(A_1 \wedge \neg A_2 \wedge A_3) \vee (B_1 \wedge B_2 \wedge B_3) \vee (\neg C_1 \wedge C_2)$

- Each proposition can be converted to CNF and DNF

History of SAT

- SAT competition (2002~)
www.satcompetition.org/
- Prime factor (decimal)
 - ✓ 15-16 digits, *ok*
 - ✓ 34 digits, *no*



MiniSat Page - Microsoft Internet Explorer

ファイル(F) 編集(E) 表示(V) お気に入り(A) ツール(T) ヘルプ(H)

戻る 進む 検索 お気に入り メディア

アドレス(D) <http://www.cs.chalmers.se/Cs/Research/FormalMethods/MiniSat/> 移動

Google G miniSAT 検索

The MiniSat Page

by Niklas Eén, Niklas Sörensson

Main

MiniSat

MiniSat+

SatELite

Papers

Authors

Links

INTRODUCTION

MINISAT is a minimalistic, open-source SAT solver, developed to help researchers and developers alike to get started on SAT. It is released under the MIT licence, and is currently used in a number of projects (see "Links"). On this page you will find binaries, sources, documentation and projects related to MINISAT, including the Pseudo-boolean solver MINISAT+ and the CNF minimizer/preprocessor SATELITE. Together with SATELITE, MINISAT was



ページが表示されました

インターネット

Pre-compiled binaries...

[MiniSat_v1.14_cygwin](#) - Cygwin/Windows binary.

miniSAT example

The screenshot shows a terminal window with the following content:

```
$ miniSAT test.cnf test.ans
===== [MINISAT] =====
LEARN
Clauses Literals Lit/Cli |
=====
0 0 0 % |
=====
67 /sec)
267 /sec)
267 /sec)
0.00 % deleted)

CPU time : 0.015 s

SATISFIABLE

mizuhito@miranda ~/Lecture/Mogi070824
$ more test.ans
SAT
-1 -2 3 0
```

A text editor window titled "MeadowNT.exe@MIRANDA" is open, showing the contents of test.cnf:

```
c *comment lines*
p cnf 3 4
1 2 3 0
-1 2 0
-2 3 0
-1 -2 -3 0
[--]-(Unix)-- test.cnf
```

Red arrows point from the following text boxes to the terminal output and editor content:

- Number of variables**: Points to the value 3 in the header line of the editor.
- Number of clauses**: Points to the value 4 in the header line of the editor.
- Representing**: Points to the clause $x_1 \vee x_2 \vee x_3$ in the list of suggested clauses.
- Result:**: Points to the assignment $x_1 = 0$ in the terminal output.

Representing

$x_1 \vee x_2 \vee x_3$
 $\neg x_1 \vee x_2$
 $\neg x_2 \vee x_3$
 $\neg x_1 \vee \neg x_2 \vee \neg x_3$
(0 is the terminator).

Result:

$x_1 = 0$
 $x_2 = 0$
 $x_3 = 1.$

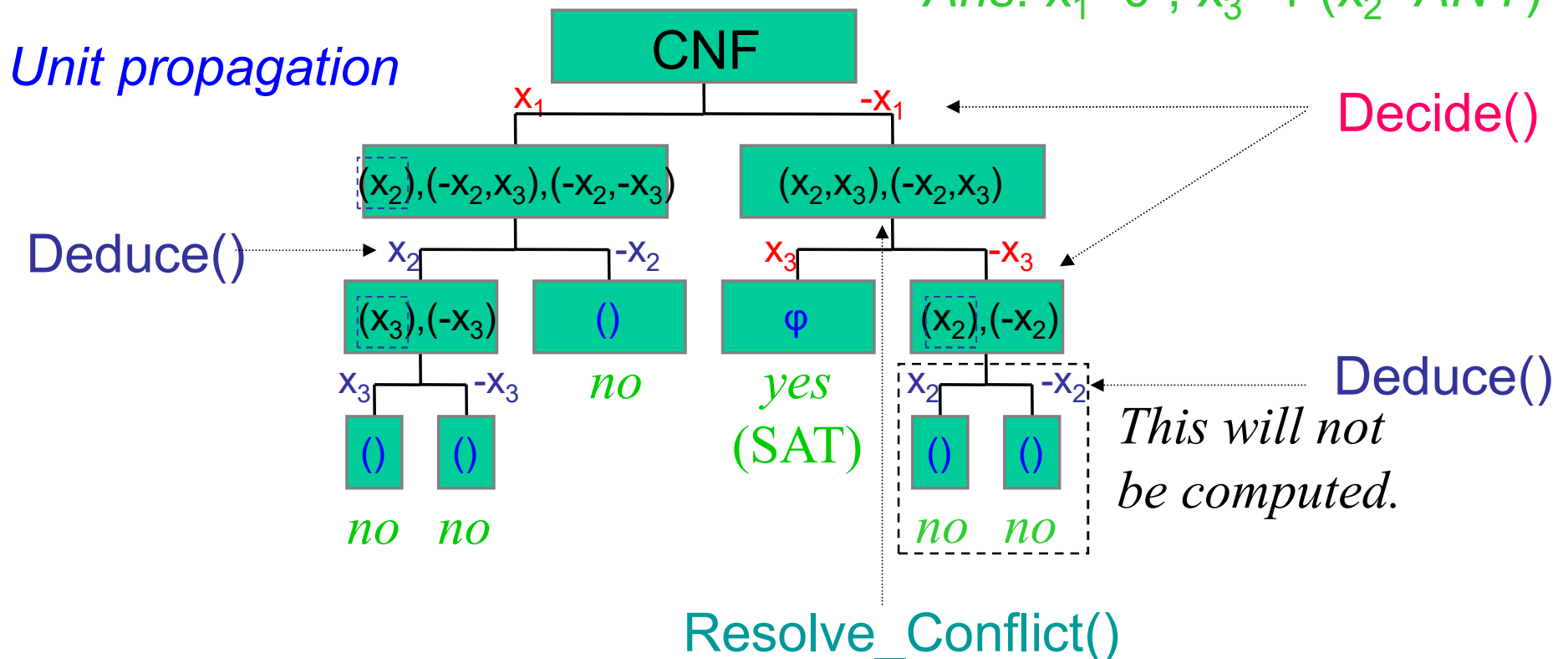
CNF suggested form

A Basic SAT algorithm (DPLL)

Example. CNF: $(x_1, x_2, x_3), (-x_1, x_2), (-x_2, x_3), (-x_1, -x_2, -x_3)$
which means

$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

Ans: $x_1=0$, $x_3=1$ ($x_2=ANY$)



Implementation design for SAT solver

- More than 90% of execution time is by “Deduce()”.
 - ✓ Parallel execution on “Deduce()”.
- Randomization and probabilistic nature.
- Boolean Constraint Propagation (BCP) design.
 - ✓ Implication graph + Non-chronological backtrack.
 - ✓ Conflict-driven learning. (Forget, restart)
- Efficient data structure
 - ✓ Two watched literals

Alternative: GSAT (Schoning, FOCS99)

- Probabilistic algorithm (n -variables for k -SAT)
 - ✓ Initial valuation α : $\{0,1\}^n$ is randomly generated.
 - ✓ Check whether each clause in a CNF is *true* by α .
 - ✓ If there are clauses that are *false* by α , choose a variable from one of them and reverse it as α' .
 - ✓ If, after repeating $3n$ times, still not satisfied, then re-generate α randomly and repeat the procedure.
- **Theorem.** If a CNF is satisfiable, GSAT almost always finds an instance after $2^n(1-1/k)^n$ times.
 - ✓ Suitable for parallel execution.

Schoning, U., A probabilistic algorithm for k -SAT and constraint satisfaction problems, FOCS99, pp.410–414

GSAT example

Example. CNF: $(x_1, x_2, x_3), (-x_1, x_2), (-x_2, x_3), (-x_1, -x_2, -x_3)$
which means

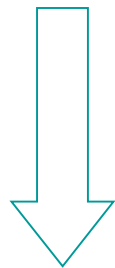
$$(x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

- $n = 3, k = 3.$

A) Randomly generate, say $x_1=1, x_2 = 1, x_3 = 1$

B) $\neg x_1 \vee \neg x_2 \vee \neg x_3$ is not satisfied. Then, flip one, say $x_2 = 0$.

C) $\neg x_1 \vee x_2$ is not satisfied. Then, flip $x_1 = 0$ (SAT)



If not satisfied, it will continue 9 times, and then return to A)

CNF

x1 v x4

x1 v x3 v x8

x1 v x8 v x12

x2 v x11

x7 v x3 v x9

x7 v x8 v x9

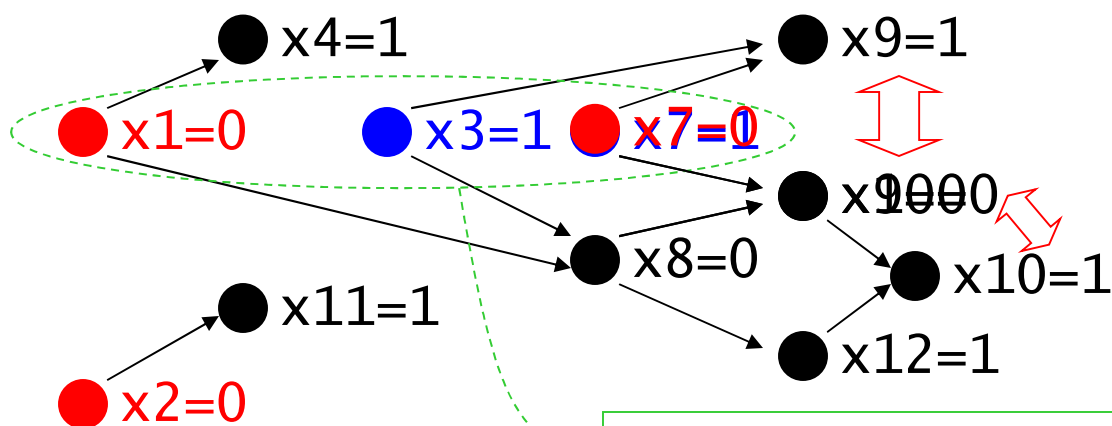
x1 v x3 v x7

x7 v x8 v x10

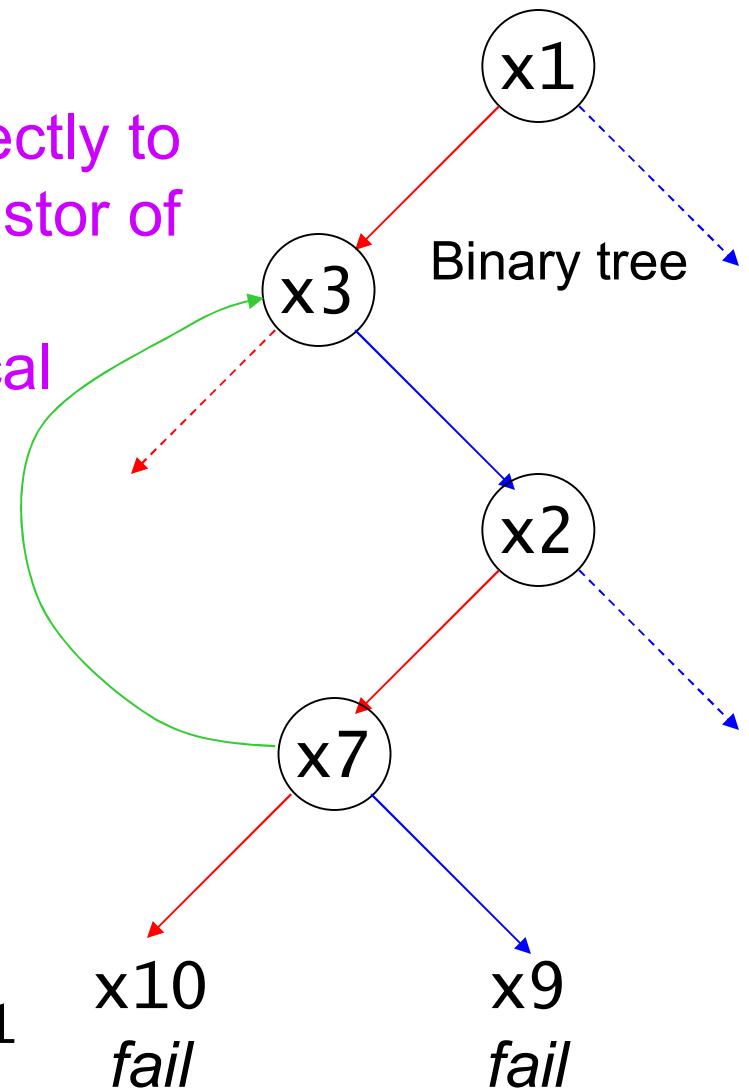
x7 v x10 v x12

BCP by Implication graph

When backtrack occurs, jump directly to the nearest ancestor of implication.
(non-chronological backtrack).



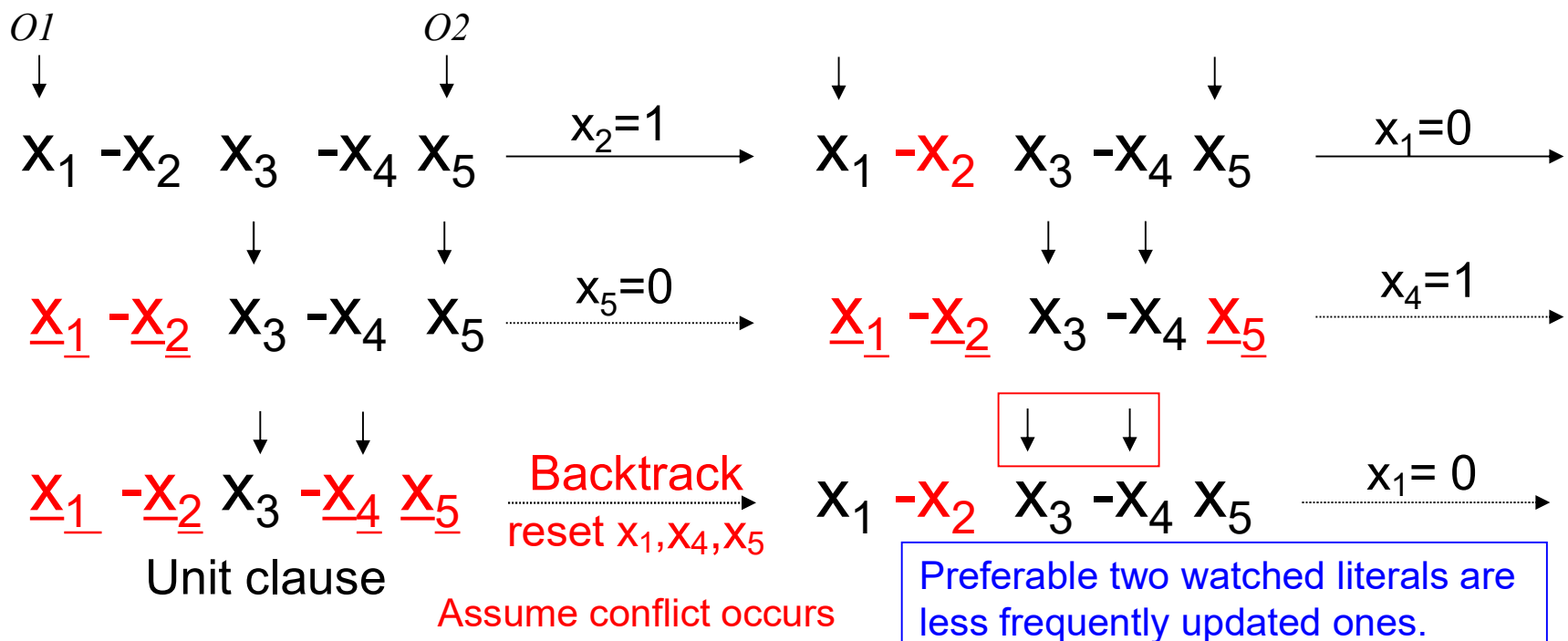
Implication graph



Conflict driven learning: $x1 \vee \neg x3 \vee \neg x7$

Two watched literals

- Observe two uninstantiated literals in each clause
 - ✓ Pointers to “*Two watched literals*” $O1(\pi)$, $O2(\pi)$
 - ✓ Suspend instantiation to a clause π , until either $O1(\pi)$, $O2(\pi)$, or both becomes *false*; then, pointers shift to uninstantiated literals.



Recent heuristic improvements

- Choice of boolean variables in “Decide()”
 - ✓ Weights on variables and statistical updates
 - More weights on variables in learnt clauses (VSIDS and variants, e.g., in Chaff, miniSAT)
 - ✓ Escape of local optimal points
 - Quick restarts (32 decide()’s) and randomization
- Learnt clauses pruning
 - ✓ Choice of the sources of failed search.
 - By implication graphs (dominators)
 - ✓ Too many learnt clauses are inefficient.
 - Forget learnt clauses (at most 500 clauses)

Decision levels and dominators

CNF

$$C_1: \neg x_1 \vee \neg x_2 \vee x_{13}$$

$$C_2: \neg x_3 \vee \neg x_5 \vee x_6$$

$$C_3: \neg x_5 \vee \neg x_7$$

$$C_4: \neg x_6 \vee x_7 \vee x_8$$

$$C_5: \neg x_4 \vee \neg x_8 \vee \neg x_9$$

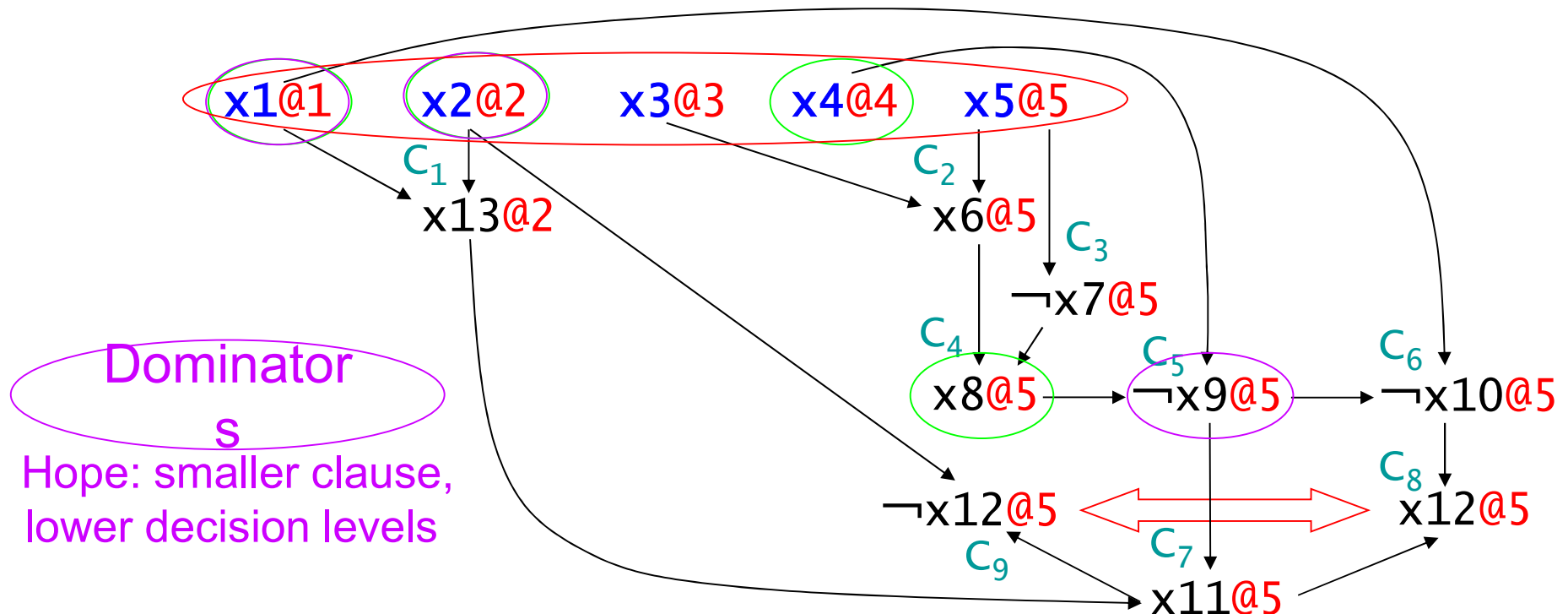
$$C_6: \neg x_1 \vee x_9 \vee \neg x_{10}$$

$$C_7: x_9 \vee x_{11} \vee \neg x_{13}$$

$$C_8: x_{10} \vee \neg x_{11} \vee x_{12}$$

$$C_9: \neg x_2 \vee \neg x_{11} \vee \neg x_{12}$$

Assume: Decide() assigns 1 in the ascending order

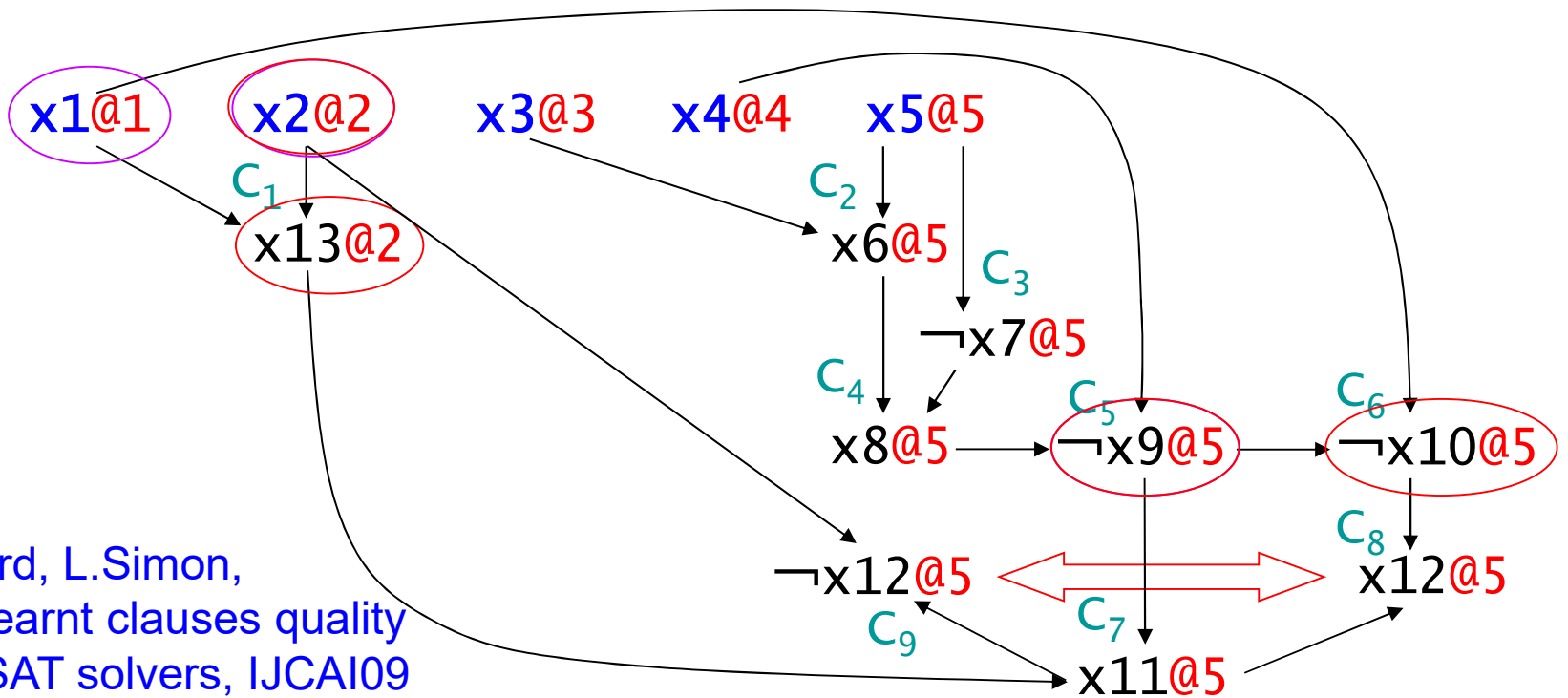


Another choice principle of learnt clauses

- Learnt clause: $\{x1@1, x2@2, \neg x9@5\}$ leads failure.
 $\checkmark \neg(x1@1 \wedge x2@2 \wedge \neg x9@5) = \neg x1 \vee \neg x2 \vee x9$,
 which consists of three decision levels.

Clauses in 2 decision levels, Generalization of a clause of length 2

- Glue clauses: $\{x_2@2, x_{13}@2, \neg x_9@5, \neg x_{10}@5\}$.



Glucose
G.Audermard, L.Simon,
Predicting learnt clauses quality
in modern SAT solvers, IJCAI09

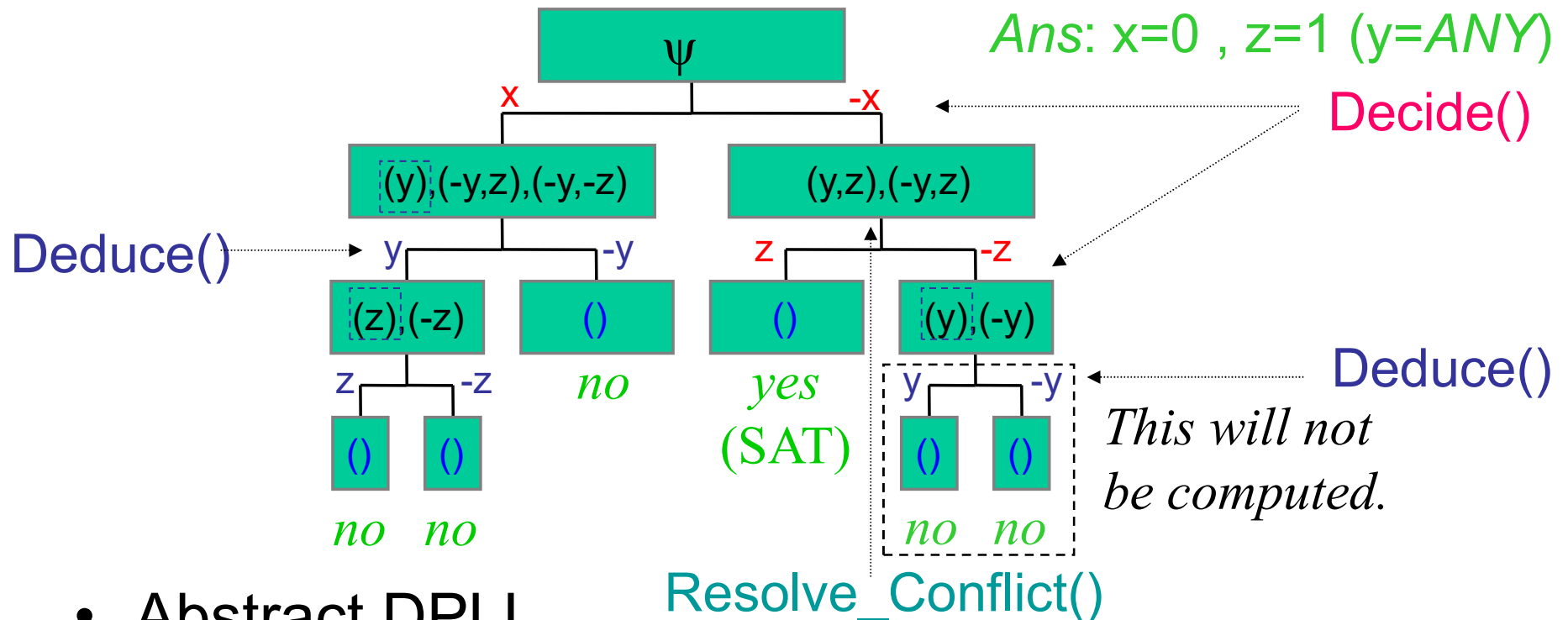
Two views of DPLL

- Abstract DPLL
 - ✓ To describe algorithm (except for data structure, e.g., two watched literals)
 - ✓ Implication graph is presented as derivation.
- DPLL as resolution
 - ✓ When generating interpolant (last lecture).

Abstract DPLL

Formalize DPLL

Given ψ in CNF: $(x,y,z),(-x,y),(-y,z),(-x,-y,-z)$



- Abstract DPLL

- ✓ M: a partial model (partial truth valuation)

- ✓ F: CNF to be solved

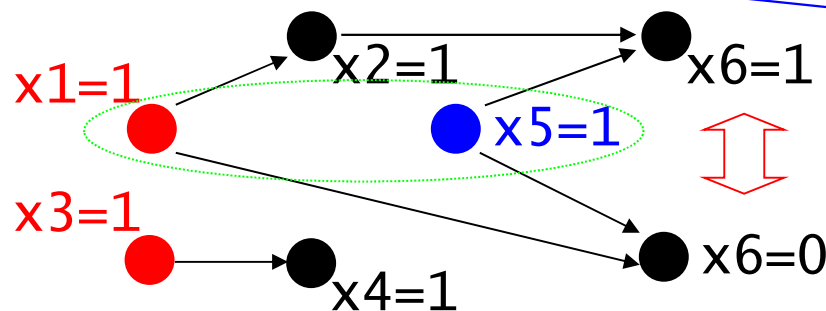
- ✓ $M \parallel F \rightarrow M' \parallel F'$, initially $\varepsilon \parallel F$

Abstract DPLL for SAT

- **Decide.** $M \parallel F \rightarrow M \text{ /}^d \parallel F$
✓ if $/$ or $-/$ occurs in F and $/$ is undefined in M
- **UnitProp.** (deduce) $M \parallel F, C \vee l \rightarrow M \text{ /} \parallel F, C \vee l$
✓ if $M \models \neg C$ and $/$ is undefined in M
- **Fail.** (UNSAT) $M \parallel F, C \rightarrow \textit{fail}$
✓ if $M \models \neg C$ and M contains no decision literals.
- **Backjump.** (resolve) $M \text{ /}^d N \parallel F \rightarrow M \text{ /}' \parallel F$
✓ If $\exists C \vee l'$ s.t. $F \models C \vee l'$, $M \models \neg C$, l' is undefined in M , and l' or $\neg l'$ occurs in F .

Example of abstract DPLL of SAT

$\varphi \parallel -1v2, -3v4, -5v-6, 6v-5v-2$ (Decide)
 $\rightarrow 1^d \parallel -1v2, -3v4, -5v-6, 6v-5v-2$ (UnitProp)
 $\rightarrow 1^d, 2 \parallel -1v2, -3v4, -5v-6, 6v-5v-2$ (Decide)
 $\rightarrow 1^d, 2, 3^d \parallel -1v2, -3v4, -5v-6, 6v-5v-2$ (UnitProp)
 $\rightarrow 1^d, 2, 3^d, 4 \parallel -1v2, -3v4, -5v-6, 6v-5v-2$ (Decide)
 $\rightarrow 1^d, 2, 3^d, 4, 5^d \parallel -1v2, -3v4, -5v-6, 6v-5v-2$ (UnitProp)
 $\rightarrow 1^d, 2, 3^d, 4, 5^d, -6 \parallel -1v2, -3v4, -5v-6, 6v-5v-2$ (Bkjmp)
 $\rightarrow 1^d, 2, -5 \parallel -1v2, -3v4, -5v-6, 6v-5v-2 \dots$



$M = 1^d, 2, 3^d, 4, 5^d, -6$
 $C = -1, \quad I' = -5$
 Then, $M \models \neg C$
 $F \models C \vee I'$

Rules for conflict driven learning

- **Learn.** $M \parallel F \rightarrow M \parallel F, C$
 - ✓ if all atoms of C occur in F and $F \models C$
 - ✓ E.g., $1^d, 2, -5 \parallel -1 \vee 2, -3 \vee 4, -5 \vee -6, 6 \vee -5 \vee -2, -1 \vee -5$
- **Forget.** $M \parallel F, C \rightarrow M \parallel F$
 - ✓ if $F \models C$
 - ✓ Remove some of too much learnt clauses.
- **Restart.** $M \parallel F \rightarrow \varepsilon \parallel F$
 - ✓ Partial results are stored as learnt clauses.
- DPLL terminates if they do not repeat infinitely often.